Self-Navigation System

Duckietown Report

2026-03-13

Authors:
Kamen Kanev
Dylan Wei Jie Neoh
Vuk Trpkovic

TUM
2026

Contents

1 | Abstract 3
2 | Introduction 4
3 | Hardware and Software 5
3.1 | Hardware Setup 5
3.2 | Software Environment 5
4 | Existing Perception Modules 5
5 | Navigation System 6
5.1 | Mapping the environment 6
5.2 | Path planning 7
5.3 | Motion Control 8
6 | Controller application and GUI tools 9
7 | Conclusion 10
Bibliography 11

1 | Abstract

This report presents a self-navigation system for Duckiebots that enables autonomous point-to-point navigation using map-based path planning. Users can create maps manually or generate them from images through road layout detection. After selecting start and destination points, an optimal path is computed using the A* algorithm and translated into lane-following and intersection-turning behaviors. The system integrates global planning with local vision-based control to enable reliable navigation in Duckietown environments. Additionally, rescue tools were developed for manual navigation, camera testing, and calibration when standard system utilities fail. Experimental results show improved navigation reliability and enhanced robustness during development and operation.

2 | Introduction

Autonomous navigation is a fundamental capability in mobile robotics, requiring robots to perceive their environment, plan routes, and execute safe motion along complex road networks. Educational robotics platforms such as Duckietown provide a simplified yet realistic environment for developing and testing autonomous driving algorithms, including lane following, intersection handling, and path planning. While many systems focus on local navigation behaviors, enabling reliable point-to-point navigation across an entire road network remains a challenging problem.

This project aims to develop a self-navigation system that allows a Duckiebot to autonomously travel between two user-defined locations within a Duckietown map. Instead of manually specifying navigation commands, the user selects a start and destination point on a map interface. The system then computes an optimal route using the A* path planning algorithm, which efficiently determines the shortest path within a graph-based representation of the road network.

To enable flexible deployment, maps can either be manually created by the user or automatically generated from images through road layout detection. The resulting path is translated into a sequence of navigation actions that combine global path planning with local vision-based lane following. When the robot reaches intersections, turning decisions are derived directly from the planned route, allowing the Duckiebot to follow complex road structures autonomously.

In addition to the autonomous navigation pipeline, a set of rescue and diagnostic tools was developed to support system development and recovery. These tools enable manual navigation, camera testing, and calibration in situations where the built-in utilities fail or require additional debugging capabilities.

The goal of this project is to provide a practical and robust navigation framework for Duckiebots that integrates map-based path planning, autonomous road following, and system recovery tools. The proposed approach improves usability during development while enabling reliable point-to-point navigation within the Duckietown environment.

3 | Hardware and Software

3.1 | Hardware Setup

The experiments were conducted using the Duckiebot platform from the Duckietown robotics framework. The robot used in this project was the DB21J Duckiebot model, which includes a differential drive base, motor drivers, and an onboard camera for visual perception. The platform was assembled from the standard Duckiebot kit before development began.

The robot was equipped with an NVIDIA Jetson Nano, which provides increased computational capability for running perception and navigation algorithms. The camera module was calibrated after assembly to ensure reliable visual input for the lane following and detection modules.

3.2 | Software Environment

Development was carried out in a native Linux environment running Ubuntu 22.04. Docker, the Duckietown Shell (DTS), and the Robot Operating System (ROS) were installed and used to configure the Duckiebot, run perception modules, and test system components as described in the Duckietown documentation [1]. The self-navigation and rescue tools were implemented in Python, utilizing several supporting libraries for tasks such as path planning, graphical interfaces, and system control [2].

4 | Existing Perception Modules

The perception modules used in this project, including **lane following**, **AprilTag detection**, and **object detection**, are based on implementations provided by the course tutors in the *Safe Navigation project* [3]. These modules were used as the foundation for the system, as the built-in Duckietown components occasionally failed or required additional debugging.

- **Lane following** enables the Duckiebot to remain within road lanes using

  camera-based detection of lane markings.

- **AprilTag detection** provides

  environment markers that can be used for localization and intersection recognition.

- **Object detection** allows the robot to identify relevant objects or obstacles during operation.

5 | Navigation System

Autonomous robot navigation is commonly divided into several stages that allow a robot to move from its current position to a desired goal. In many navigation systems, this process consists of three main steps: mapping the environment, planning a path, and executing the motion required to follow that path. First, a representation of the environment is created so that the robot can understand the available roads or obstacles. Next, a suitable route between the start and goal positions is determined. Finally, control commands are generated that allow the robot to physically follow the planned route.

5.1 | Mapping the environment

Since mapping was not the primary focus of this project, a static mapping approach was chosen instead of dynamic mapping. Implementing a full dynamic mapping system would require additional sensors and localization techniques, similar in concept to GPS-based triangulation, allowing the robot to detect its surroundings and update the map in real time. Due to time and tool limitations, this approach was not pursued.

Instead, the system uses static maps of the environment. These maps can be created by uploading an image of the Duckietown layout, which is then processed by a Python script that generates a graph representation of the road network for use in later navigation stages. Additionally, a custom map editor was developed, allowing users to recreate existing maps or design new ones manually. These maps can then be used as input for the navigation pipeline, which leads to the next stage: path planning.

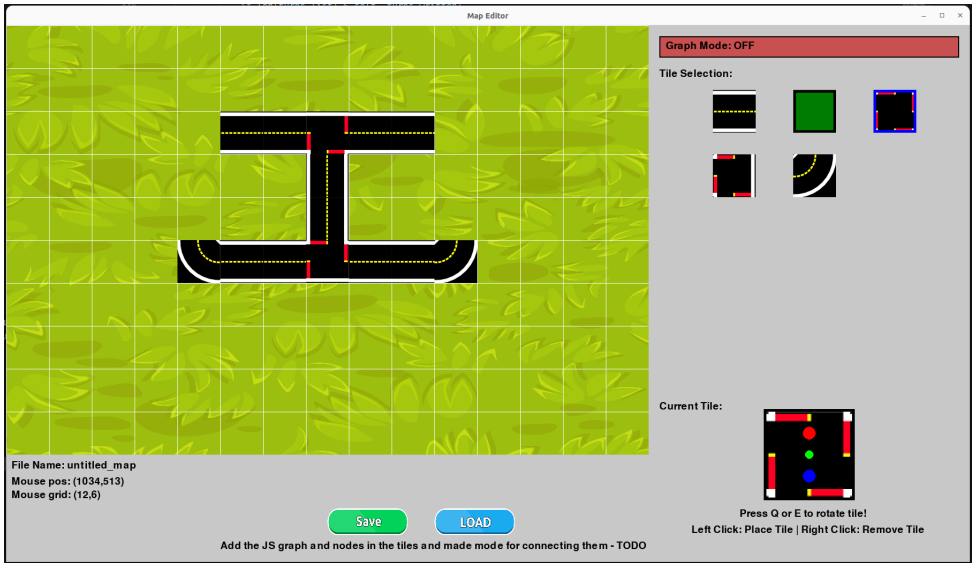

Figure 1 : Map editor

5.2 | Path planning

The first step in the path planning stage was selecting a suitable planning algorithm. A variety of algorithms exist for robot navigation, including grid-based, graph-based, and sampling-based approaches. Since grid-based methods can be less efficient for structured road networks, we focused on either a sampling-based or a graph-based algorithm. Two candidate algorithms were considered: Rapidly-exploring Random Trees (RRT) [4] and A* [5]. Both algorithms were implemented in our project repository [2] and tested during development.

Initial experiments were conducted using the RRT algorithm. While it proved effective in generating feasible paths, it required significant tuning and the addition of numerous obstacles to ensure that the generated paths remained within the correct lanes of the road network. Because of these limitations, the approach was not ideal for the structured map environment used in this project. However, sampling-based algorithms such as RRT can be more suitable for problems involving dynamic environments or obstacle avoidance, where their ability to explore the space more freely becomes advantageous.

As a result, the final system uses the A* algorithm for path planning. The algorithm was adapted to operate on a graph representation of the road network instead of a traditional grid. Nodes generated from the map representation are connected by weighted edges, allowing the algorithm to compute the optimal path between two points in the graph based on traversal cost.

5.3 | Motion Control

Once the map has been generated and a path planning algorithm has computed a route, the final step is enabling the differential drive robot to execute the path. To achieve this, the robot must remain within its lane and correctly navigate intersections. Lane following is therefore used to keep the Duckiebot aligned with the road while moving between intersections.

To determine the correct direction at intersections, additional processing is performed after the path is generated. From the planned path, a sequence of turns is extracted and stored in a turn array that indicates which maneuver the Duckiebot should perform at each intersection. During operation, the robot detects red stop lines using the camera, which indicate the presence of an intersection. When such a line is detected, the robot stops and executes the corresponding turn from the array before continuing along the lane.

This process continues until all turns in the array have been executed, indicating that the robot has reached its final destination. In addition, object detection is used to identify obstacles in the environment. If an obstacle is detected, the Duckiebot stops and waits until the path is clear before continuing. This behavior provides a simplified demonstration of concepts similar to Level 3 Advanced Driver Assistance Systems (ADAS) used in modern autonomous vehicles.

6 | Controller application and GUI tools

To integrate and demonstrate the developed components, a graphical user interface (GUI) application was implemented in Python. The application provides a controller that allows the user to select from a set of pre-created maps or create a new map using the built-in map editor. After selecting a map from the left-side menu using the arrow keys and the Enter key, the selected map is displayed in the main window shown in Figure 2.

Within the map view, the user can select two points representing the start and destination locations. Once these points are chosen, the A* path planning algorithm runs automatically. If a valid path is found, the resulting sequence of turns is stored in a binary file, and the generated turn array is also displayed in the lower-left corner of the application.

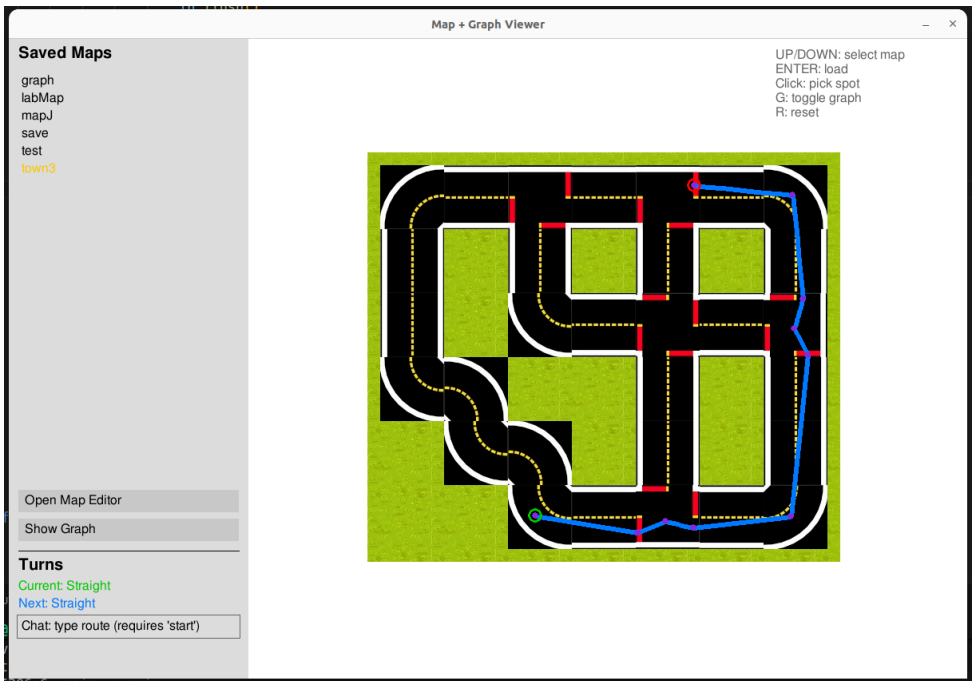

Figure 2 : Main application

The interface also includes additional tools to assist development and visualization. Users can toggle a view that displays the underlying graph representation of the map, as well as activate the map editor to modify or create new maps.

In addition, a simple LLM-based interface was integrated into the application. This feature allows the user to describe the desired route using natural language after specifying a starting point. The system then interprets the description and attempts to generate a corresponding navigation path for the Duckiebot.

7 | Conclusion

This project implemented a map-based self-navigation system for Duckiebots using A* path planning combined with vision-based lane following and automated intersection handling. The system allows users to select start and destination points on a map, enabling autonomous point-to-point navigation. Support for manually created and image-derived maps increases system flexibility, while additional rescue and diagnostic tools improve robustness during development and operation. The resulting framework provides a practical solution for reliable navigation and experimentation in Duckietown environments.

Bibliography

[1] D. Project, "Duckietown Documentation." [Online]. Available: https://www.duckietown.com/

[2] K. Kanev, D. W. J. Neoh, and V. Trpkovic, "Duckiebot Rescue Tools." 2025.

[3] E. Mehmedova, I. Panayotov, and E. Mehmedova, "Safe Navigation for Duckiebots." 2024.

[4] S. M. LaValle, "Rapidly-Exploring Random Trees: A New Tool for Path Planning," technical report, 1998.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 1968.